

Summary

Action Items

- ☐ Create sequence diagrams showing workflows for new bugs vs bugs with incremental changes ▶ ▶
- ☐ Add RAG implementation to the architecture block diagrams ▶
- ☐ Redesign UI homepage to display auto-discovered bug list instead of manual entry ▶ ▶ ▶
- ☐ Implement dashboard with bug status indicators and critical bug counts ▶
- ☐ Add filtering options for projects, source systems, and bug status ▶ ▶
- ☐ Implement pagination for bug listings ▶ ▶
- ☐ Add similarity scoring indicators for related issues showing percentage match ▶ ▶
- ☐ Enhance LLM matching to include bug title, description, logs, timestamps, customer impact, troubleshooting steps, and comments ▶ ▶ ▶ ▶ ▶
- ☐ Create token settings page allowing users to configure API credentials, base URLs, and port numbers for each bug tracking system ▶ ▶
- ☐ Develop detailed use case diagrams and sequence diagrams breaking down the system into smaller components ▶ ▶
- ☐ Create implementation plan showing how work will be distributed among team members ▶
- ☐ Add Knowledge Base support portal to external sources ▶
- ☐ Upload all meeting recordings and documents to GitHub and share link ▶
- ☐ Submit checkpoint review materials using this recording ▶ ▶

Redesigned Architecture Overview

- Team presented updated High-Level Design based on previous feedback ▶

- System starts with engineer submitting bug ID/issue, which is authenticated using JWT tokens stored securely in token vault ▶
- Token vault also stores API keys and credentials for external systems like Jira, GitHub, and Bugzilla ▶
- FastAPI serves as entry layer, validates JWT tokens, and pushes requests to Kafka topic for fault tolerance and scalability ▶ ▶

Agent Pipeline Architecture

- Task orchestrator acts as central coordinator managing all agent communication ▶
- Agents are completely isolated from each other and communicate only through orchestrator ▶
- **Phase 1:** Context agent runs alone, fetches full bug details from source system, retrieves linked customer case notes, and extracts keywords for downstream agents ▶
- **Phase 2:** Cross-system fetch agent and enrichment agent run in parallel after phase 1 completes ▶
- Cross-system fetch agent searches Bugzilla for related bug reports and GitHub for related issues using keywords from context agent ▶
- Enrichment agent extracts keywords, builds SQL queries, searches Confluence in read-only mode, and returns KB articles, runbooks, and past incident reports ▶
- AI synthesis agent combines outputs from all three agents and sends to LLM ▶
- LLM returns structured JSON with suggested severity, probable root cause, recommended next steps, and source references ▶

External System Integration

- Connector registry pattern replaces hardcoded if-statements for extensibility ▶
▶
- Each external system (Jira, Bugzilla, GitHub) has individual adapter class in registry ▶

- Adding new systems only requires updating connector registry without changing pipeline logic ▶
- Redis layer with 5-minute TTL cache prevents redundant API calls for repeated queries while maintaining data sovereignty ▶

UI Design Feedback and Required Changes

- Current design requires manual bug ID entry, which needs to be eliminated ▶ ▶
- Tool should auto-discover all bugs requiring triage and display them in list view ▶ ▶
- Implement collapsible/expandable rows showing triage details without requiring navigation ▶
- Add search functionality within list view and multiple filter options for projects and source systems ▶
- Include status indicators showing severity, priority, and whether bug is closed ▶
- Current history tab shows right concept but needs complete listing with pagination, not just recent items ▶ ▶
- Create tree view for correlated bugs using one bug system ID as primary key with others as child nodes ▶

Bug Correlation and Matching Requirements

- Related issues must show similarity scores indicating percentage match confidence ▶
- Implement LLM-based matching instead of keyword search alone ▶ ▶
- Matching should consider multiple fields: bug title, keywords, tags, status whiteboard, customer impact statement, severity, serial numbers, timestamps, descriptions, troubleshooting steps, maintenance changes, workarounds, log information, crash dumps, performance data, and all engineer comments ▶ ▶ ▶
▶ ▶ ▶
- Similarity scoring tiers: >95% excellent match, >90% good match ▶

- Some bugs have direct references (like CFI customer-found issues) that should be utilized when available ▶

Data Management and Caching Strategy

- System needs mechanism to detect and fetch incremental changes from dynamic bug tracking systems ▶ ▶
- Should fetch latest data from source systems without storing everything in own database ▶
- Temporary caching in PostgreSQL allowed during pipeline execution for specific bug and associated bugs ▶ ▶ ▶
- Cache must be flushed after processing completes ▶
- Should not create data bottlenecks by storing all bug details permanently ▶

UI Result Dashboard

- Four main sections displayed after pipeline completes ▶
- Bug context panel shows core details (title, priority, status, component, assignee) from context agent ▶
- Related issues panel displays similar bugs from other systems with direct links to original sources ▶
- Linked context panel shows KB articles and runbooks from enrichment agent ▶
- AI summary panel consolidates all information showing suggested severity, root causes, and recommended steps ▶
- Pipeline progress visible through status indicators and progress bars ▶ ▶

Token Configuration Requirements

- Need settings page for users to configure API tokens for each bug tracking system ▶ ▶
- Must allow multiple URLs per system (e.g., different Jira projects) ▶
- Users should provide base URL including port number and API token for each ▶

- Backend should support JSON configuration for all setting details ▶

External Sources

- Current external systems: Jira, Bugzilla, GitHub, Confluence, Customer Portal ▶
- Approved to add Knowledge Base support portal for documented workarounds and known issues ▶ ▶

Next Phase and Checkpoint Review

- Major architecture approved and ready to move to implementation phase ▶ ▶
- Study phase considered completed ▶
- Team should use this recording as checkpoint review submission (due Jan 14-18) ▶ ▶
- Need to develop detailed use case and sequence diagrams before implementation ▶
- Must create plan for breaking down work and distributing among team members with clear integration points ▶
- Rajesh (another mentor) requested recording as they couldn't attend ▶

Notes

Transcript

You have a project?

Let it come, it's okay. Did you join the group?

I can't see anyone. I can see 4 people. I think he's going to SS Thank you.

Sorry. Join me, Orozco.

Sir is asking me to wait for 5 minutes.

I am feeling very tired.

I didn't mean to Wi-Fi and I had Microsoft Teams Wi-Fi.

It doesn't work well on the net. into wi-fi Manish is also going to join. He saw the message and talked to me.

Too good.

Can't we have a hot pot for the whole gig?

Um... My wife has her own nerds. Phone nerds.

He is trying but now My background noise is very low No, I didn't. No. I need soup.

Good evening, sir.

Yeah, hi, good evening. Good evening, sir.

Good evening.

Is chat disabled in free version of Teams? Why it doesn't come up? It says chat is turned off for this meeting. If you have to share something over chat, then it will not be possible.

So when we create link, like we have that free world, we are using that free version only. So that option is not shown anywhere, like how to enable chat for all. Okay. That's the issue.

So if there's anything we'll message on WhatsApp then. Okay. As part of this meeting, if there's anything I'll share that. Yeah. Sure. See if anybody else would join. Give another minute.

Have you added the other mentors also?

In this Yes sir, I sent an invite to all but Rajesh sir replied that he won't be available today. So he asked us to record this all. Like the other two mentors, like no response received. Okay, so we are recording it from our side. Yes. Okay.

That's the case. Let's get started then.

Okay, sir. Anuj, can you please share the screen? Sure.

Is my screen visible? Yeah. Yeah. So based on the previous feedback, we redesigned our HLD. We'll now walk you through it. So it all starts when the engineer submits bug ID or a bug issue and the request hits our tool. Firstly, it is authenticated using JWT tokens. These are signed tokens that are like securely stored in our token vault and any other request that the engineer makes on our tool this token is attached to it so that we don't have to validate it again and again

Also, when we are reading data from external systems like Jira, GitHub and Bugzilla, we also need to have a way to securely store their credentials.

So we are using the token vault here, where all the API keys and the credentials are securely stored. After that, the requests hit the entry layer where we are using fast API as our back in. So it is a front. It is like a front door for all our request. All the request gets sent through this. It validates the JWT token and pushes the request to a Kafka topic. We are using Kafka here so that if anything were to happen to any part of our system, the bug ID or the bug issue that we are reporting gets preserved and it does not get lost.

It mainly serves the purpose when we are scaling our pipeline and making it fault tolerant.

So this is how the request is coming in. Now I'll take you through what happens when that request basically reaches the orchestrator. So one important thing before anything else is like in the agent pipeline, whatever agents are being shown, they are not communicating with each other directly. Basically all the communication that is happening between them is managed by the task orchestrator, which is a kind of coordinator acting, sitting in the middle.

the one who is managing everything like from assigning tasks to the agents collecting their outputs and deciding what flows where so the basically we can say that agents are completely isolated from each other and they do not have one agent has no idea about the existence of other agents so this agent pipeline runs in different phases like the first phase is about the context agent running alone it needs to run alone because its output is required by the other downstream agents it needs to run alone like the main purpose the input that it gets is the bug id and the source system exactly what the engineer types in and gives us the input and then it fetches the full bug details from the source system like the source system let's say the source system is zero so it fetches all the details like title bug title description components uh priority everything and then it fetches the link customer case notes from the customer portal and what it sends as an output The orchestrator is the complete bug details plus the extracted keywords that will help the other agents like the cross system and enrichment to make search from the other systems and give the output.

Now this is the phase one. The moment the phase one is completed, the orchestrator is the one who will start the phase two by firing the cross system

fetch agent and the enrichment agent. Both of these agents can run in parallel. The input that is given to the cross system fetch agent is are basically the keywords which are received from the context which agent which are passed by the orchestrator to it and then it searches for the other systems like bugzilla for related bug reports and github for related issues and open full requests and it outputs back to the orchestrator the related bugzilla reports with status and severity the github reports the github issues all with the direct link back to the original system now handing over to om to explain the further of this pipeline.

So now we have an enlistment agent written intelligently extract important keywords from the book title and description, build a SQL query and search its confluence in read-only mode. It then returns the most relevant knowledge base articles, runbooks and past incident reports. So now AI Synthesis Agent takes output from all three previous agents like the context agent for primary bug details, the cross system agent for related issues.

and the enrichment agent for KP articles and confluences. Now it combines all of this and you can comprehensively from and send it to the LLM. The LLM then returns structured JSON output containing suggested severity, probable root cause, recommended next section, and source reference in the copy. Confidences Now that this chance forms, cater draw data into clear and actionable prize insights. Oh.

It's a bunch. Okay, so now we'll discuss... Okay, I'll do it.

Okay, go with it.

So now we'll discuss how our system is connecting to external sources like Jira, Bugzilla and Github? So, the naive approach is to write FL statements like if this query is found in Jira or if this query is found in Bugzilla, we can call their API every time. But in this case, we have to manually change our logic each time we are adding a new system like let's say ServiceNow or any other bug tracking tool.

So what we are doing to overcome this, we are making a connector registry which contains adapters for each of those external system we are connecting. Each system acts as an individual class in our connector registry and the agent makes all the request through this connector registry to all the external systems. So every time we add a new system or we need to change something, we do not need to hard code any of the logic in here.

We just need to add a system in our connector registry and the pipeline is unaffected from there. We also have a Redis layer with a five-minute TTL cache, so that if a query is repeatedly hitting our system, The same query is repeatedly hitting our system. It does not waste resources or API tokens when we are triaging it again and again. It can fetch the previous results from memory. We are just storing it for 5 minutes and after that this is erased from the system so it like prevents the data sovereignty problem that we were facing last time.

So after this we can discuss the UI portal in detail. So like if there is a change in the bug the way like it goes through it continuously keep changing right so yeah there is no mechanism here something like a rag you may need to use right like you have to affect the incremental changes that have been happening. Whenever the query that comes. like changes as in on the source system like something like Is a life cycle right it keep changing Jira all these things are dynamic so well You report one part one aspect and then tomorrow if you see there would be more progress on that Are there more of similar issue or the some fix might have been identified? So each time it has to look up and fetch the latest data.

So Thank you. Like last time we were trying to keep track of every change that was happening. But for that we needed to like store some data into our postgres. Without storing you should be able to fetch the latest. Every time the query comes, without storing you should be able to fetch the latest data from the data source itself. But you shouldn't be storing everything in your own database. Rather, you have to detect if there is a change.

There's an incremental change, right? From what you have reported previously, Yes. about fetching the incremental changes that has happened. If it's a new bug, then what is the workflow? If it is, let's say a bug which had some incremental changes, then what is the workflow? And then how it correlates to this architecture and which part of the architecture takes care of the refresh part. We also prepared a UI mockup so can we proceed to show that maybe some of your doubts can be resolved in that.

Yeah, but UI mockup is one aspect which is how this shows up. But do make those sequence diagrams which I talked about. Yes, sir.

Yes, sir. Yes, sure.

And also call out like if you are utilizing RAG, then that need to show up here in one of the block diagrams. Sure, sir. Yeah. Anuj over to you please.

Yes sir. Is the screen visible? Yes. So now I will walk you through the UI of a bug triage tool and how an engineer sees the bug. So basically we start on a new bug triage step. This is the entry point of the tool where the engineer will select the source system.

Yeah, before you start, go away from this. So typically a bug triage tool should have discovered all the bugs that require triage and there should be a list. This view should be a list. It should not be a text box like this where you expect user to find out what is the bug number and enter it here. So that's the first comment. So it should be a self-discovered tool. There should be some setting somewhere where if you specify the Jira or whatever, sorry not Jira specifically, if you set a particular project for a particular product rather, then it should be able to discover.

the list of those bugs like something like that is pulling live from the repositories like the last 10 bugs It should pull and it should be showing the list and that through the list user should be able to see as it is like on the maybe you could come up with a sort of a collapsible expandable rows. Where if I expand then it should show the triage details and all that without expecting user to enter and then click on try triage.

got it sir yeah yes sir so we need to completely remove this thing that i need to clear up like this uh engineer entering a bug id manually that thing we need to completely eliminate yeah it could be a search in the list view yeah specific bug id if user want to search then he will enter that in the search box There should be multiple filter options based on different projects and other things. Filter options should be provided and also filtering based on different source system like you mentioned here, there's a drop-down, so that should become a filter.

And then the automatically all the listing of the bugs that need to be trashed. And then also the status indicator of that particular bug, whether it is a critical bug, a severity, all of that, whether it is closed or not. So based on the current status, there should be enough UI indicators which will easily allow the user to see at a glance what is the state of the overall system. And there should be a, on top you can have a dashboard also to give you the critical what are the critical count of the critical bugs And the ones which has triage information And the ones which are...

and it doesn't have sufficient information or whatever, like the categorizations that we have. So accordingly, I create a dashboard on the top, and then there could be a list view below it, which can be expanded at this level. I'm just giving you an

idea. But overall message is that it should discover all the bugs and also provide sufficient filters to navigate through the UI. Anuj, can you once go to the history tab?

Something like this in case of list you were talking about. a few bugs in the homepage and then we can individually try it if you want to. It should be completely available. The entire list should be available and there should be pagination options. across all the bugs. There should be filters here for each column. and this should be able to filter based on the project and the bug source. that whether it is Jira or from different systems, I should be able to filter out also.

Then for a particular triage ID or you could create your tool specific ID and for that you can correlate multiple bug systems, bug tracker IDs and group them. and show it again. So that would be good. Yes. Take one of the systems primary key as a primary key. One of the bug systems ID itself as a primary key and then rest of the associated ones can be within the as a child nodes. So you can create a tree view of that sort.

then that would be a very good representation. But it should have complete listing. Either you should be able to auto scroll or there should be pagination options. Okay, so... We'll take that into account. our next iteration. Yep.

So, sir, should I continue like after all the changes if we have the search bar after searching any particular bug, what all will be processed, how it will be processed? So, yeah, page. Okay. So basically if the user doesn't find its bug in its dashboard also it will basically Select its source system either boxy log it up and there is case ID then it will click run right as soon as it click the system transition occurs like this and pipeline occurs So basically it ended very fast, but I had taken a screenshot So here we can see that different agents being triggered and some running sequentially and federally Which reduce our overall tries time?

I will not go deep down the agent path already as it has been already explained by my teammates you The UI provides clear visibility into the pipeline through status indicators, progress bars, and allowing users to understand what all is happening. So once a pipeline completes, we move into the result dashboard. At the top we can see that we have bug ID, its description, its overall time taken. There are majorly four sections.

The first section is bug context panel which provides the core details of the bug. This comes from the context agent. It includes the title, the priority, the status,

component, assignee report and so on. In the second panel we have related issues. This data comes from the cross system fatigin. It shows similar or related issues like bugzilla data. For example if we have searched for Jira then the Bugs would be showed from boxilla or GitHub Also, we have these related issues include the Direct Link.

We have the direct link here. So if the engineer wants to explore further, they can simply click on this and navigate to the original issue for more detailed information. Third is our link context. It comes from the enrichment agent. It includes supporting resources such as knowledge base articles or runbooks. It will allow engineers to directly access the troubleshooting guide and relevant documentation.

And lastly is our AI summary panel. synthesis agent so what it will do is it will consolidate all the gathered information you know and show us at a single uh panel for example provide the suggested severity level a brief and root causes recommended steps also this help engineers make informed decision quickly without going from multiple sources and so on So to summarize this complete UI brings multiple system into a single workflow.

Yeah, so the thing is like to see how are we correlating the issues across various bug tracker systems. So that should be there should be an indicator in terms of how did this AI or the LLM matched it and what is the there should be as some score. which will indicate what is the percentage of match, LLM match. Right, similarity score.

Kind of a coincidence. Yes, sir.

How did you match the related Github IDs? Is it based on the linking that is there within the system itself or are you expecting that let's say within Jira and Bugzilla are there linking available or is it mentioned that this particular bug is the associated one or are you using LLM to try and search? Most probably there won't be direct matches like that because these are different bug tracking systems. able to use LLM and then do matching.

Based on the whatever information that is available within the bug details of each tracker. So maybe you could look at logs, support bundles that are there, the issue description. or the event logs. So you will have to do LLM matching across all the, all this. across which you should be able to find similarities and you have to give a similarity score. Whether it's an excellent match or is it a fair match.

More than 90% could be a... 95% could be an excellent match. Above 90% could be a good match and things like that. should be able to call it. Like initially we were using the keyword search from our original bug ID. So we can enhance it using LLM reasoning, like searching across the description of the other bugs. So we can definitely implement that. Take into factor not only description, but the other textual information that is there, steps to reproduce.

See, you have to figure out what is the use case based on which the bug is occurring, because the use cases may be different. Error could be same. So you have to go deeper into the way bug lifecycle of a particular issue. is tracked and what are the things I can give you a sample data in terms of how a typical bug what does a typical bug will have? based on our Vaxzilla Jira tracker, right? Yes, sir.

Can make a note of it as a brief description. Thank you. Okay. Thank you. Sir, can you repeat the last part? It was a bit unclear. I think network issue. Yeah, what I was trying to say is that like what are the various fields that are there? In a bug based on which you will be able to look for the similar matches. So first one is the bug title itself. Right? Then you have the keywords. Some of the bugs will have keyword fields and tags and keywords that will be status whiteboard.

There'll be customer impact statement, customer summary, Then who's the customer, primary customer, the severity, serial number of the device array or whatever. then it can have I know where the dead 10 timestamp when the issue occurred. So if you match this date and timestamp, it means that occurrences have that you that each of these different trackers are talking over the same timestamp. That way.

Getting my point right? Okay, sir. Got it. In the description of the customer issue, summary of the troubleshooting steps. And what are the recent maintenance or changes that have happened? And what is the request to engineering? Any workaround provided if there is outage. Advancing outage issue solved or not. Right, and log file, if there are any time differences between the hardware components in the data center.

Right? Yes, sir. And location of available logs and the log information Right. And whatever supporting tools that are there, which can give more information about let's say there is a panic, then what are the crash dumps available? Then is there a textual description of the analysis of the crash that has happened? Or if there's a

performance issue, what is the performance capture data? So all of this, information is all textual information for you to match.

Thank you guys.

and all the comments that are there in the system.

which have been made by various engineers to trash them manually. So all of that gives you the necessary context to build the similarity score. which you can then get the AI insights also, populate more information useful information in the ai insights and also correlate the bugs across various systems if it is available if there's a direct match some of them do have let's say for example if there's a cfi it's a customer found issue then there will be a reference to the actual bug id itself so wherever available we could directly utilize and try to fetch that if it is configured so there should be a page where users should be able to provide all these uh api Thanks.

Right? So, each of those system create a, I see that you have a token settings link there. So it should take all the API tokens for each bug triad system.

and again, all of that and correlate based on all the fields that I have just mentioned. So then that will be a good Yeah, you know, a thing to demonstrate. Okay, sir. We'll iterate on it in our next meeting. Yeah, this is what I was talking about. I understood like what, This is what I was trying to say. All the API tokens that will be required, for connecting to a particular cloud, there should be a way to provide that by the user and configure it.

Like so.

Yeah, you can is there a mockup for the edit? No sir, that is not yet implemented. What I am trying to say is that though you say Jira cloud, so there will be multiple URLs. Like each project could be within Jira itself, there could be different URLs that will be, you know, that need to be contacted. So you should be able to provide the user to enter the base URL. Okay. Including the port number and also the API token for each of those.

So create such screens that user can take such, give provide the such inputs and also your backend system should have maybe a JSON or something through which I can specify all those details, setting details. Okay, so. Yeah, yeah. Oh. Oh. We had a question sir. Yeah. Yeah, see you at the diagram. Yes, sure.

Now is my screen visible?

Yes, so how agents are working, we have agents and stateless. So if we are fetching data from external sources, we are putting, giving that data to our solver stator. And the question arises here, Stop.

Sorry, your voice is breaking. Oh Let me ask that. Sir, what he is trying to ask is when we are giving context to our multiple agents through our pipeline, So some of the data has to be transferred between them. So To do that, we need to store it for a limited amount of time in our Postgres SQL. So is that all right? to do for a few minutes or so. when we are running the pipeline. Yeah, there can be a sort of a cache. which you can persist for a limited time.

Yes. For a particular session. And then there can be in your diagram you are not showing really the interactions between the agents. So is it by design you have things because agents should be able to work with each other We're sharing the data right across agent. So that is really coming out or you can explain how? Like we are managing the agents using task orchestrator. So whatever the context agent for design.

Then if the agents and data are exchanged, then it has to go through the task orchestrator. Yes, so we need to like have the data for a limited amount of time in the Postgres SQL, that's what we were asking. Is that all right? But it should not be like a mammoth of data. It should be basically based on data the bug or the like it's one specific bug related data, right? Not.

Yes, the entire context of that particular associated bugs, it's all right.

Yeah, there should be no bottleneck sort of a data store where you are necessity. I mean, where you have to store all the bug details. So it is only the a particular bug which is under process and it's associated with bugs from different trackers. Those you can always have a cache which you can... but remember to flush it once the processing is completed.

Oh, yes sir. Yes. Yep.

Like the major architecture diagram that is fine. So we can move on with implementation. Come up with the use case level, use case diagrams and the sequence diagrams in terms of what is the life cycle of a bug trash. And what are the things that are, so I want you to, this is a high level diagram, you know, This is one diagram, but I want you to come up with what are the use cases that are there and the tag of smaller diagrams related to that and showing various interactions.

So that will give more That will be more helpful when you start to break down the things into smaller pieces and how you will be chunking the entire stack here and you have to distribute this among yourselves, right? So come up with a plan in terms of how you will break this down and also what are the interactions involved so that you know between which smaller teams or between which person you have to Work on the design related to the integration part and things like that.

So work on all of those aspects before getting into the actual implementation.
Okay, sir.

Yes. Thank you.

Okay, so good presentation. So anything else?

Are the external sources fine? Do we have to add something else in the external sources or remove something?

Where is it, the external source? The external system, the last node. API Gateway The last node at the bottom Okay. Do we need to add more or less? Jira, Parxila, GitHub, Confluence, Customer Portal. Yeah, this is good enough. Yeah, these are the ones, Confluence articles. GitHub is there. Yeah, GitHub and Confluence is where we have all our... repo and also the documentation related to that. There can be a knowledge base support Hotel which will have knowledge base about existing cases and all that So that you can add maybe.

And is it fine to query for the KDR?

Yeah, KB articles will be there, right? Where it would have details related to the workarounds, documented workarounds, and no issues and things like that. can be a support knowledge base you can call it yes yeah yeah It's fine for the technical questions.

We wanted to ask about the mid-project presentation. which we were also supposed to present between 14th to 18th. and information regarding that.

What is it? The checkpoint something they are saying is about that? Yes, yes. Yeah, I mean you can share this recording. This itself can be a checkpoint. So with That I think we can call this as a study phase completed. You already presented the understanding of all of the EIML aspects that are required in the previous sessions. Right. All of that you summarize it and share the recordings and say that this can be taken as a checkpoint review.

That's the reason all these recordings and these documents, everything, you please upload it in the GitHub and provide the link to them. Well, it is following up on it.

Sure. Good to meet you. Yeah.

Anything else? Oh.

No. OK.

Thank you, sir. Good job.

Thank you. Thank you, sir. Bye-bye. Nice. Sorry guys. Someone is... And from my side too. Yeah